TP TUBES (anonymes et nommés)

1 - TUBES ANONYMES

1.1 - Test anonymous pipes with the linux console. To do this, type for example the command:

```
# pstree | more
```

Explain the flow of this command by the console knowing that the symbol "|" represents an anonymous tube.

Exercise 1: Reading in a tube without an editor

Write a program that tries to read from a tube without an editor.

Exercise 2: Writing in a tube without a reader

Write a program that attempts to write to a tube without a drive. What signal is generated in this case. Install a handler that intercepts this signal

Exercise 3: Communication between processes using a tube

Write a program that creates a child process. Father and son share a hit. The father writes in order an operator, an operand 1 and an operand 2. The child will have to read this information to perform the operation (addition, multiplication, subtraction or division) and display the result.

Operands 1 and 2 are integer values while the operator corresponds to the character '-', '+,' * 'or' / '. When the father has nothing more to generate, he sends the character '#' to mark the end of writing to the pipe.

Exercise 4:

The goal is to make two processes communicate with each other using anonymous pipes. Write a program in which:

- The parent process creates a child process and sends it a message.
- The child process reads the message and then responds to the parent process.
- The father process waits for his son to respond before ending.

Exercise 5:

The goal is to make the parent process communicate with these two child processes.

1. Write a program in which:
   a) the parent process creates two child processes "pid1" and "pid2" and then writes the 26 letters of the alphabet into a tube (each lowercase letter is followed by the same uppercase letter: aAbBcC….).
   b) child processes read letters one by one and then display them.
2. Modify the program to synchronize the reading of child processes.
   a) The child process "pid1" reads lowercase letters.
   b) The child process "pid2" reads uppercase letters.

Exercise 6:

Write a C program that creates and array of 100 random integers with values ranging from 0 to 99. It reads an integer num from the user, searches for this integer in the array, and prints if it is found inside the array or not

The code to create an array of random numbers:

```
int data[100];
int i;
```

```
for(i = 0; i < 100; i++)
        data[i] = rand() % 100;
```

In order to speed up the search, the program creates a child process

The parent:

Searches for num in the first half of the array [0, 50[(excluding 50)
If it is found, it prints "Number found by parent at index …"
Otherwise, it reads the result of the child from the pipe, if it is a negative number, it prints "Number not found...". Otherwise, it prints "Number found by child at index …".

The child:

Searches for num in the second half of the array [50, 100[
If it is found, it writes its location (index in the array) to the pipe and terminates.
If it does not find it, it writes -1 to the pipe

Exercise 7:

1- Write a program in C allowing two parent and child processes to transmit the date and time to each other via an anonymous pipe. Use a maximum of printf in order to "trace" the execution of the program. Here are a few steps to help you:

The father
- creation and testing of the anonymous tube
- creation and testing of his son
- waiting for the end of the thread
- tube reading
- date display on screen

The son
- 2 second nap
- the son redirects its standard output to the tube
- execution of the "date" process
- death of the son

Useful functions:
pipe, fork, dup2, execl, wait, read, write ...

Questions :
• what happens if we delete the waiting phase of the child by the father (with wait) ???
• what happens if we use a printf after the dup2 primitive in the child ???

Exercise 8:

In the same spirit as the previous exercise, create a C program capable of making two parent and child processes dialogue. The son will ask you for the message to send to his father (scanf), and the father will display it on the screen (after having read it in the tube of course).

2 - TUBES NOMMES (ou FIFO)

Exercise 2.1

We will test the use of named pipes using a few unix commands.

```
# mkfifo mon_tube
# ls -li
# file mon_tube
```
(gives the genre of the file. Try with a text file, an executable, an image ....)

`# echo " the weather is nice " > mon_tube` (the process is suspended)
Open a second console:
`# cat < mon_tube` (magie ... magie ...)
Try the manipulation in the other direction (i.e. "**`cat < ...`**" before the "**`echo`**".
Conclusion ...
We can delete the named pipe using the command:`# unlink mon_tube`

Exercise 2.2

Write two programs in C: **writer.c** and **reader.c**. These 2 programs will exchange a message via a named pipe created by the program ecrivain.c
writer.c: usage #. / writer nom_du_tube "message to send"
drive.c: usage #. / drive tube_name
Advice: for the program reader.c try to use an array of characters which you will initialize before reading it in the pipe.
Useful functions:
mkfifo, unlink, open, read, write

Questions :

- what is the variable PIPE_BUF worth (see course)?
- check the rights of the named pipe. Do they agree with those switched to the primitive mkfifo? Why ?

Exercise 2.3

Write two other programs in C: client.c and server.c. These will communicate data through a named pipe using a structure like:

```
struct struct_donnees
{
  pid_t pid_client;
  char message[50];
};
```

client.c will ask you to enter a message and send it and its pid to the server.c program. On its side, server.c will display the data received on the screen (message + client PID). It is the server.c program which will create the pipe. The server.c program will use a loop to read several messages, and it will be necessary to provide an exit from this loop when it receives the "END" message.
For your experiments you can try to launch several clients, set a time delay on the server side...

Exercise 2.4- On the same principle, write two programs **client_fichier.c** and **server_fichier.c** with the following variants:
- **client**_fichier.c will be used like this:
`#. / client_fichier tube "message to send"` (so no more scanf ())
- file_server.c will display the data on the screen and write it to a `data.txt` file

For your tests you can use a shell script to launch several client_file programs as below (for example):

```
#!/bin/sh
./client_fichier tube le
./client_fichier tube chat
./client_fichier tube mange
./client_fichier tube la
./client_fichier tube souris
```